



Nectar

THE UNDERSTANDING LAYER

THE APIARY · LEGION CODE × ACTIVELOOP

How much time do your engineers lose because AI reads the wrong code?

Your developers' agents find code by file name, so in a real codebase they read the wrong files, answer confidently about the wrong thing, and hand the search back to the engineer. Nectar describes every file by what it does and stores that on Deeplake, so agents find the right code by meaning, once, for the whole team.

What does 'the agent read the wrong file' actually cost us?

Every wrong file is a wasted turn you pay for and a confident answer your engineer has to catch and correct. Nectar points agents at the files that actually matter, so they spend tokens on the right code the first time.

How does one engineer's understanding reach everyone?

Nectar's descriptions are a portable registry: build the understanding once and the whole team inherits it when they pull the project, with no re-indexing per person. A new hire's agent is useful on day one instead of week three.

Where does this live, and is it secure?

On Deeplake, reached only by the local daemon, with teams and projects isolated at the storage layer and secrets never exposed. The descriptions are versioned, so the map of your codebase is auditable, not a black box.

Why is meaning-based recall a moat, not a nice-to-have?

Deeplake matches on what code does, at scale, so agents find the right file even when nobody named it well, which is most real code. That accuracy compounds across every question every engineer asks, all day.